ContractKV: Certified Prompt Compilation for Transactional Prefix Reuse in Multi-Tenant LLM Services

Bin Zhou and Xi Cao, *Member, IEEE*

***Abstract*—In multi-tenant LLM services, repeated documents often follow changing questions or observations, leaving substantial reusable content beyond the reach of native prefix caching. Reordering that content creates a physical prefix, but its behavior and shared visibility must be governed together. We present ContractKV, which makes a dependency-valid compiled prefix an evidence-bound service object. CALC exposes legal stable prefixes; execution-scoped token-path and termination checks govern behavioral admission; Prefix-Selective Transactional Promotion (PSTP) atomically publishes eligible complete KV blocks; and Service-Template Certificates bind authorized reuse to a versioned lifecycle and amortize qualification. On six source-disjoint HotpotQA/RepoBench templates with task-preserving 16K inputs and zero tool slack, Qwen3-8B, Mistral-7B and Llama-3.1-8B achieve 1.619×, 1.109× and 1.070× complete-service throughput. All 1,296 admitted compiled future pairs reproduce raw output token IDs under greedy decoding. In deployment-template controls, sharing qualification reduces verifier calls from 125 to five and unhidden verification exposure from 25.05 to 3.24 ms/request. At a 25-tenant, 80%-load deployment point, 500-ms SLO goodput rises from 0.173 to 0.831 requests/s. Together, these results demonstrate how scoped admission, atomic publication and template-level cost sharing turn structural repetition into useful shared service state.**

***Index Terms*—Large language model serving, KV cache, prefix caching, behavioral certification, transactional publication, multi-tenant services.**

I. Introduction

A retrieval service places a user's question and conversation history ahead of a long, mostly immutable document collection. Another authorized tenant asks a different question over the same documents. The stable context is reusable at the application level, but native prefix caching stops matching at the first changed token; it cannot recover the documents' downstream KV. This ordering is not hypothetical: a fixed revision of an Azure RAG demonstration places history and question before retrieved sources [1]. Our measured task-preserving traces likewise have 98.36–98.52% stable-body tokens, yet recover only 0.030–0.034% of that stable body through native prefix hits (Section II).

An obvious remedy is to move stable documents before the changing fields. That does not make the result deployable. *First*, the reorder must preserve data, causal and positional dependencies. *Second*, a dependency-valid reorder can still change an autoregressive answer, so a legal layout needs behavioral evidence before use. *Third*, a speculative candidate can become visible in the shared cache before qualification completes, and an otherwise valid reuse may cross a stale template or unauthorized scope. A cache hit, a behaviorally admissible layout and a published shared block are three different facts.

Prior mechanisms solve important but different parts of this problem. PagedAttention and RadixAttention implement exact-prefix reuse [2], [3]; Prompt Cache provides reusable schemas [4]; CacheBlend and EPIC explore cross-position execution [5], [6]. Application-aware scheduling and relational row/field reordering expose further reuse opportunities [7], [8]. ContractKV addresses the complementary service decision: binding transformed state to applicable behavioral evidence, authorized consumers and an atomic publication boundary (Section II).

Our claim. *A structurally reusable KV prefix becomes a useful multi-tenant service object when legal compilation, execution-scoped behavioral admission, transactional publication and authorized qualification amortization are governed together.* We realize this service object through four coupled mechanisms: legal compilation, scoped admission, transactional publication and template-level qualification sharing. Figure 1 follows an example request from raw order to an admitted, physically shared prefix.

Our contributions connect each mechanism to a measurable service property:

1) **CALC: dependency-aware prefix compilation.** CALC constructs legal stable-prefix layouts and derives their identities after full-prompt retokenization, exposing reuse opportunities that native contiguous matching cannot reach (Sections IV-B and VI-B).
2) **Scoped behavioral certification.** Token-path and termination evidence binds a candidate to its declared execution scope. Free-running task pairs measure behavior preservation, and controlled candidate challenges establish the value of admission over layout alone (Sections IV-C and VI-C).
3) **PSTP: transactional shared-state publication.** The engine keeps candidate KV private until PASS and publishes only certified exact-prefix full blocks. Rejection, interruption and binding failure preserve the shared publication boundary (Sections IV-D and VI-D).

B. Zhou is with Beijing University of Chemical Technology, Beijing 102218, China (e-mail: 2025200800@buct.edu.cn).

X. Cao is with Beijing University of Chemical Technology, Beijing 102218, China (e-mail: caoxi@buct.edu.cn).

Corresponding author: Xi Cao.

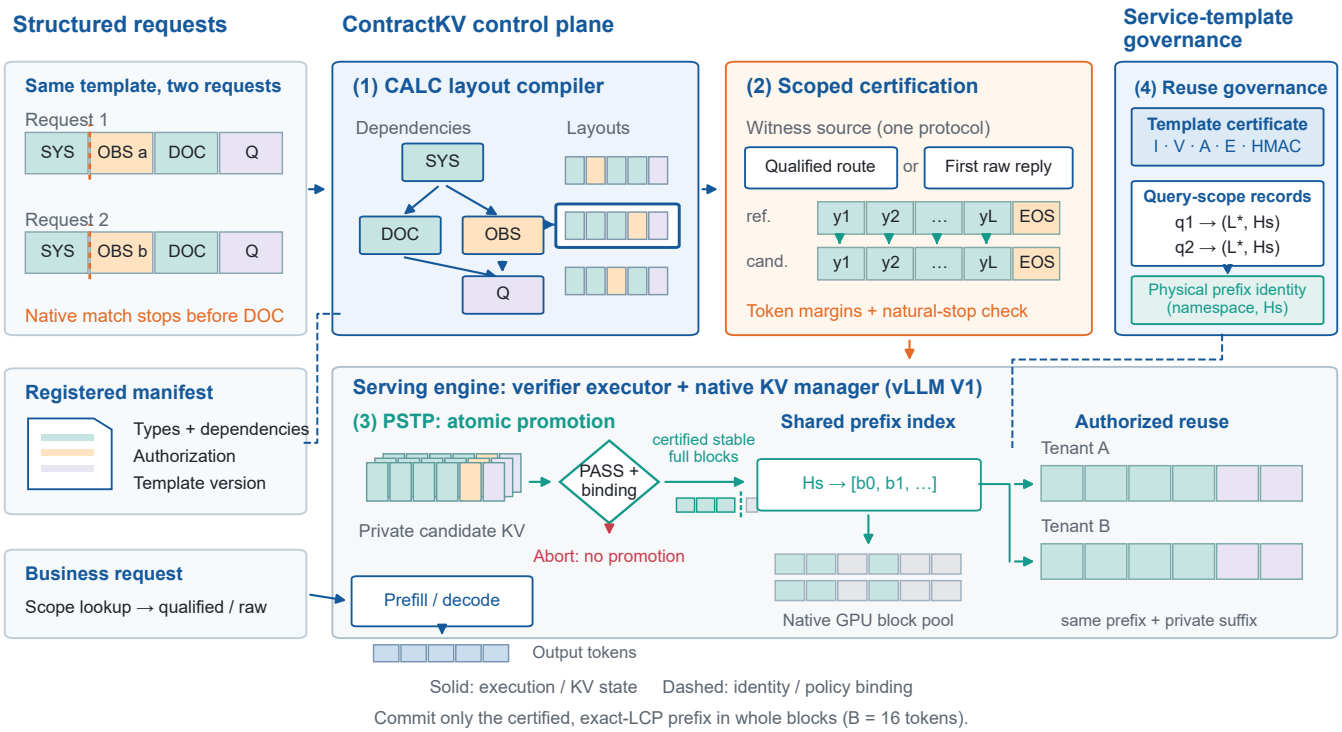


Fig. 1. ContractKV makes a compiled stable prefix an evidence-bound service object. CALC repairs a structural prefix gap; the evaluated witness protocols supply scoped token-path evidence; PSTP commits eligible state only after PASS and a valid binding. Authorized tenants of the same template share the prefix and keep changing suffixes private. Template certificates and client query records associate applicable qualifications with the physical prefix (Table III). The commit cut excludes incomplete blocks and private suffixes.

4) **Service-Template Certificates.** A versioned, authorization-bound service object lets tenants share a qualified physical prefix and its evidence cost. Fixed-layout controls isolate qualification amortization, while arrival and density studies measure its service value (Sections IV-E and VI-D1).

Task-preserving zero-slack serving achieves 1.619×/1.109×/1.070× complete-service throughput on Qwen/Mistral/Llama, retaining raw output IDs in all 1,296 admitted future pairs. Deployment-template controls reduce verifier calls 25-fold and unhidden exposure 7.7-fold; an 80%-load point gains 0.658 SLO-compliant requests/s. Table III defines the witness, scope and cost unit of each protocol. These results show how *admission, publication and qualification sharing turn a reuse opportunity into a service asset*.

II. Background and Motivation

A. The Gap Between Repetition and Prefix Reuse

A causal language model first computes prompt states during prefill, then generates output autoregressively. Native prefix caching reuses a leading sequence whose tokens and cache identity match a resident entry [2], [3]. For fully rendered prompt $\mathbf{t}$, let $\mathcal{P}(\mathcal{R})$ be the token prefixes represented by resident state under the matching cache identity. The block-aligned reuse opportunity is

$$k^* = \max\{k : \mathbf{t}_{1:k} \in \mathcal{P}(\mathcal{R})\},$$
$$k_B(\mathbf{t}, \mathcal{R}) = B\left\lfloor k^*/B \right\rfloor, \tag{1}$$

where B is the block size. Byte-identical text outside that leading sequence offers a layout opportunity. Its KV state also depends on preceding tokens and positions [2].

The Azure template in Section I establishes a realizable prefix-breaking order [1]. The following controlled traces independently measure the reuse gap; they do not estimate that application's traffic or sharing frequency.

B. Measuring the Structural Opportunity

We reconstruct stability and native reuse from actual API prompt IDs, the frozen renderer's segment spans, and scheduler-recorded prefix hits. Let n_r be the prompt length, S_r the set of token positions fully contained in manifest-declared stable body segments, and h_r the measured native hit length. Three distinct token-weighted quantities are

$$\rho_s = \frac{\sum_r |S_r|}{\sum_r n_r}, \qquad \rho_h = \frac{\sum_r h_r}{\sum_r n_r},$$
$$\rho_{s|h} = \frac{\sum_r |S_r \cap \{1, \ldots, h_r\}|}{\sum_r |S_r|}. \tag{2}$$

The first describes stable content, the second all native hits, and the third recovery of the stable body itself. The full-prompt denominator includes chat-control tokens and headings; the stable body excludes wrappers and cross-boundary tokens. Source-macro averages are calculated separately.

Table I shows substantial stable content but little native recovery in these controlled traces. An independent five-family, growing-history trace contains 3,750 actual requests and 3,375

TABLE I
MEASURED STRUCTURAL OPPORTUNITY IN THE SIX-TEMPLATE, TASK-PRESERVING COHORT. EACH ROW CONTAINS 1,620 FUTURE RAW REQUESTS ACROSS THREE REPETITIONS. RATIOS ARE TOKEN-WEIGHTED AND USE EQ. (2).

Model	Stable body ρ_s	Native hits ρ_h	$\rho_{s\|h}$
Qwen3-8B	98.520%	0.950%	0.030%
Mistral-7B	98.477%	0.985%	0.034%
Llama-3.1-8B	98.357%	1.114%	0.031%

TABLE II
CLOSEST MECHANISMS ORGANIZED BY THEIR PRIMARY REUSE DECISION.

Mechanism	Primary decision
Native APC / Radix	Does a request share a contiguous physical prefix? [2], [3]
Prompt Cache / cross-position	Which context modules or distant content can be reused or recomputed? [4]–[6]
Application scheduling	Which ordering, placement or dataflow exposes more reusable computation? [7], [8]
ContractKV	**When may a transformed prefix become authorized shared state, and how is its evidence cost repaid?**

future requests; its equal-family stable-body ratio is 94.144% and stable-body native recovery is 0.595%. The supplement reports its actual message orders, separate bounded controls and a matched four-source input comparison. Tables S-I/S-II give prepared/serving orders and measured token displacement for their declared input cohort.

The separately frozen input-position ledger contains 4,695 segment-marker observations over 240 model/session inputs. Compilation moves 64.9% of markers in ordinal order, with a segment-weighted mean absolute displacement of 118.10 tokens (Supplementary Table S-II). These are measured header-marker offsets, not a causal attribution of RoPE or attention changes.

C. From a Layout Opportunity to a Shared Service Object

Continuity leaves two service decisions: which legal layouts have evidence for the intended behavior, and which candidate blocks may become atomically visible to authorized consumers. An overbroad qualification or stale identity can affect tenants beyond the request that created the state.

The same layout is only a candidate until its behavior has applicable evidence. The service contract therefore distinguishes *admission*, which authorizes a qualified path, from *publication*, which makes eligible blocks visible, and from *residency*, which determines whether authorized requests realize a physical hit.

III. SERVICE MODEL AND DESIGN OBJECTIVES

A. Structured Requests and Registered Dependencies

A structured request $r = (s_1, \ldots, s_m)$ contains application segments. Each segment carries an identifier, original content, a stability type, and a registered content identity. We retain three types: STABLE, MUTABLE, and QUERY. Stable content belongs to an immutable template version; mutable content includes observations and changing state; query content carries the current task. A query may recur within an authorized directory while remaining outside the promoted stable prefix.

The trusted operator manifest defines segment boundaries, separators, stability, dependencies, and the sharing domain (Section VIII-B). Unregistered tenant content remains private. A segment dependency graph $G_r = (V, E, \lambda)$ associates each edge with a data, causal, or positional ordering constraint. A layout π is legal when it is a topological order of G_r:

$$(s_i, s_j) \in E \quad \Longrightarrow \quad \mathrm{pos}_\pi(s_i) < \mathrm{pos}_\pi(s_j). \tag{3}$$

All cache identities and prefix boundaries are computed from the fully rendered prompt $\mathbf{t}_\pi$, including the model's chat template. Per-segment token counts guide construction, but exact boundaries follow retokenization; concatenation can change tokenization at segment edges. Let $P(\pi)$ denote the token boundary of the leading shareable stable content. CALC provides a bounded candidate set $\Pi_L(G_r)$ containing raw order L0 and registered compiled levels. Behavioral qualification selects among these legal candidates.

B. Templates, Qualifications, and Cache Identity

A service template

$$\Theta = (V, \mathbf{S}, \mathcal{A}, \mathcal{P}, \mathcal{M}, F) \tag{4}$$

binds a version V, stable content $\mathbf{S}$, an authorized tenant set $\mathcal{A}$, a frozen policy $\mathcal{P}$, its manifest $\mathcal{M}$, and declared fan-out F. The input may interleave stable and private segments; stable-first order is a compilation result.

ContractKV uses two related identities. A *behavioral qualification* determines whether a request may use a particular compiled layout. A *stable-prefix identity* identifies the corresponding reusable KV prefix within its authorized template namespace. Several admitted query scopes can map to the same stable-prefix identity.

Execution identity binds the model, tokenizer/chat template, decoding/stopping settings, policy and runtime. Section IV-E defines the complete template and lifecycle binding.

C. Publication and Service Objectives

ContractKV aims to turn admissible structural reuse into lower complete service cost while preserving the following publication properties.

P1: Evidence-gated publication. Candidate compiled state becomes shared only after a completed verifier pass and a valid publication binding.

P2: Stable-prefix confinement. PSTP publishes complete blocks within the stable, certified, exact-match boundary. Mutable and query suffixes are excluded from this promotion.

P3: Atomic failure handling. A failed, incomplete, expired, or conflicting transaction contributes no newly promoted candidate blocks. Raw-order execution remains the service fallback.

P4: Authorization-bound reuse. The deployment control path validates template identity and authorization before admitting reuse, and separates cache namespaces across incompatible sharing domains.

These properties concern admission and publication. We assess behavior with paired free-running outputs and task scores, and efficiency with native hits and complete service wall time.

IV. CONTRACTKV DESIGN

ContractKV compiles a legal prefix, qualifies its behavior, and transactionally publishes it for authorized reuse (Fig. 1). Qualification and physical residency have separate lifetimes.

A. An End-to-End Request Example

Consider `SYS | OBS(a) | DOC | Q`, with stable `DOC` and tenant-specific `OBS(a)`. Native matching stops before `DOC` when the observation changes. CALC constructs the dependency-valid order `SYS | DOC | OBS(a) | Q` and retokenizes the complete prompt. In query-qualified serving, the first raw answer supplies witness tokens $y_1, \ldots, y_L$ and natural stop e. After token/stop checks pass, PSTP atomically publishes eligible stable full blocks. Later authorized, qualified requests reuse that prefix; an unqualified query stays raw. Compilation exposes the opportunity, evidence admits its use, and PSTP controls shared visibility.

B. Contract-Aware Layout Compiler

CALC operates on the registered dependency graph. Stable segments may be split into contiguous, character-preserving units to construct a bounded layout ladder. A stable-priority topological traversal moves only eligible units, respects dependency edges, and breaks ties by original order. For the 16-unit ladder, L1, L2, and L3 expose 4, 12, and 16 eligible units, respectively; L0 retains raw order. Full retokenization determines the actual stable-prefix boundary of each rendered candidate.

Source-disjoint development freezes allowed model–family–layout combinations. Deployment uses the registered ladder and allowed descent; query-qualified execution tries one development-fixed candidate per scope. Inherited L0 families remain raw.

The task-preserving renderer keeps essential fields and source whitespace intact (Section V). For an expanded graph $G' = (V', E')$, priority topological construction costs $O(|E'| + |V'| \log |V'|)$ per fixed candidate; model-backed qualification remains the dominant admission work.

C. Scoped Behavioral Certification

1) Witnesses and token-path checks: A behavioral witness is a completed model-generated output trajectory associated with a rendered input and execution identity. Query-qualified execution obtains it from the actual first raw business response, avoiding a separate raw generation solely to create a reference. Write its nonterminal witness tokens as $\mathbf{y} = (y_1, \ldots, y_L)$ and its natural terminal token as e; the recorded output IDs include both. The verifier evaluates the compiled prompt on the forced history $\mathbf{y}$, including termination.

For token z at output position i, define its next-token margin as

$$m_i(x, z) = \log p_\theta(z \mid x, \mathbf{y}_{<i}) - \max_{v \neq z} \log p_\theta(v \mid x, \mathbf{y}_{<i}). \quad (5)$$

The raw witness supplies its own per-position margins. The candidate must choose each witness token, and the terminal token, as its top choice. In the evaluated query-qualified protocol, the raw floor is 0.0625 and the candidate and terminal floors are 0.125:

$$\begin{aligned} m_i(x_{\text{raw}}, y_i) &\geq \tau_{\text{raw}}, \\ m_i(x_{\text{cand}}, y_i) &\geq \tau_{\text{cand}}, \quad i = 1, \ldots, L, \\ m_{L+1}(x_{\text{cand}}, e) &\geq \tau_{\text{cand}}. \end{aligned} \quad (6)$$

The raw terminal margin is also checked. Complete witness IDs, finite margins, the stop protocol, and the publication boundary are required for acceptance. The task-score threshold used in evaluation is separate from these model-level conditions.

Under identical next-token distributions, token agreement through termination gives the same greedy sequence by induction. Finite precision and execution shape condition that statement; free-running experiments test actual execution. The bound identity includes decoding, budget, stop protocol and model/runtime.

TABLE III
TWO EVALUATED QUALIFICATION PROTOCOLS SHARE THE IN-ENGINE VERIFIER/PSTP BACKEND, WITH DISTINCT REQUEST-SIDE ADMISSION RECORDS AND CHECKS.

	Deployment-qualified	Query-qualified
Witness	Compiled trajectory on a development-qualified route	First real raw business reply in the selected scope
Admission	Client checks HMAC template binding and lifecycle	Controlled-client lookup at $(\text{namespace}, q)$
Identity	Model/runtime, version, layout, policy, authorization, epoch	Bound query/stable fields, execution settings and policy; prefix association at $(\text{namespace}, H_s)$
Cost unit	$O(1)$ attestation per registered template	$O(U)$ decisions for U scopes; at most one GPU attempt per selected scope
Evidence	Registration, amortization, open arrival and density	Free-running task pairs and complete service wall

2) Qualification scope and request binding: Table III distinguishes the two evaluated paths. Deployment registration attests a representative *compiled* trajectory on a prospectively qualified model–workflow–layout route; this is deployment-route evidence, not a proof that every later raw request would produce the same answer. Query-qualified execution instead uses an actual raw business reply. Its scope key q hashes the exact query, system message, stable segments, model identity, generation settings/budget, renderer and policy.

The query-qualified experiments permit a declared metadata-only observation schema containing session identity, turn, and status, with no additional answer evidence. History and scratchpad segments are outside that schema. The shared-scope eligibility rule accepts a short, non-abstaining reference of at most 16 normalized words when it is a literal stable-context span, or when its raw minimum margin is strictly above 0.125. References failing this rule are classified as requiring an exact-observation scope; the evaluated trace declares no such repeated observations, so those requests remain raw. A further Python statement check is retained for the declared RepoBench subset. These frozen eligibility conditions define the empirical class; metadata typing and positive margins alone do not prove future invariance.

Algorithm 1 Single-shot query qualification. All work remains on the service clock.

Require: Registered Θ, request r, frozen policy
Render raw input; derive scope q; look up (Θ, q)
if an issued qualification matches this request **then**
  Check compiled-prefix identity; serve with native lookup
  return the business response
end if
Apply the pre-reference gate; serve r in raw order
if q is new and selected for qualification **then**
  Reserve q; retain the actual raw response as witness
  if witness/scope/output checks and post-reference gate pass **then**
    Verify one frozen candidate on witness tokens and EOS
    if PASS and valid publication binding **then**
      PSTP commits the eligible prefix; record qualification
    else
      Abort with zero candidate promotion; record rejection
    end if
  else
    Record the raw-route decision without GPU qualification
  end if
end if

3) Compact in-engine verification: The verifier runs on the serving model backend. Previously approved, hash-matching resident prefix blocks can be reused during verification; a one-block guard and the answer/stop checks protect the verification boundary. The executor collects the rows needed for the witness and termination rather than exporting full prompt-logprob tensors. It returns a compact decision and publication record.

D. Prefix-Selective Transactional Promotion

PSTP makes a verifier decision a shared-cache event. Candidate KV initially resides outside the shared prefix index. The state machine advances through PRIVATE, VERIFYING, VERIFIED, and COMMITTED; unsuccessful paths end in ABORTED.

Let L_s be the candidate's stable boundary, L_{cert} the authorized prompt-prefix boundary, and L_{lcp} the exact token LCP between the candidate request and its certified prompt. The promoted length is

$$L_{\mathrm{commit}} = B \left\lfloor \frac{\min(L_s, L_{\mathrm{cert}}, L_{\mathrm{lcp}})}{B} \right\rfloor. \tag{7}$$

The scheduler-thread insertion of the eligible block hashes is the publication linearization point. The corresponding KV tensors are already allocated; commit changes their visibility in the shared index rather than copying the tensors.

A non-pass, incomplete result, timeout, duplicate transaction, identity conflict, or invalid scope prevents new candidate publication. Cleanup releases private candidate resources. Raw execution can still populate the engine's ordinary cache, so the transactional property is about candidate promotion rather than freezing the entire cache domain.

After a successful commit, the native cache manager governs residency and reference counts. Its reclamation of an unreferenced block removes a physical reuse opportunity without invalidating the behavioral evidence. A later admitted request can recompute a missing prefix under the qualified layout. Section VI-E verifies this separation using actual lookup, publication, and reclamation events.

E. Service-Template Certificates and Amortization

A service-template certificate organizes the identity and lifecycle of reusable compiled state. The deployment representation is

$$C_\Theta = \langle I, V, H_s, L, P, Q, A, E, \Sigma \rangle, \tag{8}$$

I identifies the model/runtime execution, V the template version, H_s its stable content, L its layout, P the verifier policy, Q the qualification manifest, A the authorization domain, E the epoch, and Σ the authentication tag. The evaluated deployment client uses HMAC-SHA256 and checks the complete binding before selecting the compiled route.

The query client stores decisions at $(\mathit{namespace}, q)$ and prefix associations at $(\mathit{namespace}, H_s)$, allowing several qualifications to authorize one physical prefix. These client records are distinct from the deployment certificate. Changed bindings deny reuse; changed deployment objects require re-registration.

For N sessions sharing a deployment template, registration changes from $O(N)$ session attestations to $O(1)$ template attestation plus per-request checks. With T template identities and U query scopes, the controlled client uses $O(T+U)$ state and the $O(U)$ decisions in Table III. Fan-out amortizes these decisions when the authorized query directory is fixed.

F. Benefit Admission

Qualification incurs cost before later reuse. A frozen gate therefore separates investment in evidence from candidate acceptance, using remaining authorized uses, marginal prefill savings, information cost and verification cost. Its pre-reference estimate is

$$\widehat{G}_{\mathrm{pre}} = \widehat{p}_E \widehat{p}_{P|E} \widehat{R} \widehat{\Delta t} - \widehat{C}_{\mathrm{info}} - \widehat{p}_E \widehat{C}_{\mathrm{GPU}}. \tag{9}$$

After the witness is available, the gate uses conditional pass probability and full predicted GPU cost. Publication history selects a calibrated cold/warm profile; raw-native reuse is discounted. Profiles are frozen before evaluation.

The evaluated default applies a predicted waiting-budget test in cold sessions and uses expected aggregate benefit in template fan-out. We evaluate a separate length-aware waiting-budget variant in Section VI-E0a. Benefit admission can reduce qualification attempts; it never substitutes for the token-path or publication checks.

V. IMPLEMENTATION AND MEASUREMENT

A. Integration with the Serving Engine

The prototype targets the vLLM 0.10.2 V1 scheduler and cache manager. It comprises a request-side compiler and qualification controller, an in-engine verification executor, and PSTP hooks at cache-index publication. Model weights and attention kernels remain unchanged. Candidate identities are derived after full prompt rendering and tokenization. The archived performance clients use policy/template cache identities for verification and compiled requests. A separately evaluated authorization integration extends the identity to every engine route: a trusted authority validates membership and the active epoch, then derives a domain/epoch-bound cache salt for raw, fallback, verifier and compiled requests. Salt propagation

through the native block-hash chain makes physical sharing follow the authorized group (Section VI-D2).

The deployment client checks HMAC binding/lifecycle; the query client checks frozen identities and scope (Table III). Both invoke in-engine verification/PSTP. The all-route integration permits authorized raw fallback for an invalid compiled binding, but rejects expired identities before dispatch. It is measured in its own regression cohort; performance tables retain their archived clients.

Compiler and publication overheads measured in the deployment implementation are 0.389 ms mean for CALC, 0.249 ms for the residency lookup, 0.109 ms for the template-scope check, and 0.636 ms for PSTP finalization. The query-qualified studies measure their complete controller path afresh rather than importing these component timings as constants.

B. Task-Preserving Input Construction

The adapter retains each source in full when it fits the common 16K window. Otherwise a fixed head/tail rule crops optional paragraph/code-line substrings, protecting the complete question and code prefix, original order/whitespace and an omission ledger. Selection never inspects answers, outputs, scores, margins or qualification. Adjacent pieces of one source segment remain contiguous in both routes.

Both routes receive identical retained content, checked against every model/tenant/turn/layout condition, chat template and output reserve. Offline tokenization is compared with API prompt IDs. Common immutable packing is outside the clock; request rendering, planning and verification are included.

The input-construction experiment compares fixed-word-cap, whitespace-normalized packing with structure-preserving rendering under the same window, admission rules and scorer. This isolates how task-critical content affects absolute quality and admissible reuse (Section VI-E0a).

C. Execution Environment and Timing

The primary hardware is two RTX 3090 GPUs with 24 GB each. Single-model cells use one GPU; Qwen3-14B uses tensor parallelism across two GPUs in the deployment evaluation. The query-qualified environment is Python 3.10.20, PyTorch 2.8.0+cu128, vLLM 0.10.2, transformers 4.55.2, Triton 3.4.0, and FP16. Model checkpoints, tokenizers, runtime files, inputs, and policy identities are recorded per cell. The primary task-preserving cells use two concurrent requests; ten registered tenants per source describe sharing fan-out, not ten simultaneous GPU requests.

A fresh server process is used for each cell. The closed-loop client runs one tenant pair through ten rounds, then advances to the next pair. The next round becomes ready when the preceding business wave finishes; synchronous qualification performed between waves contributes to subsequent readiness delay. For request r,

$$T^{\mathrm{ready}}_{\mathrm{TTFT}}(r) = t_{\mathrm{first}}(r) - t_{\mathrm{ready}}(r). \tag{10}$$

Full service wall includes all business responses, first-use raw witnesses, planning, information and verification calls, unsuccessful attempts, waiting, scoring, logging, and final drain. Server startup, common pre-measurement infrastructure checks, configuration loading, and immutable input preparation have separately declared boundaries.

Closed-loop throughput is completed business requests divided by full service wall. The paired multiplier is the raw wall divided by the ContractKV wall for an equal request population. In the separate fixed-arrival deployment studies, SLO goodput counts responses meeting a specified readiness deadline per unit service time. Their arrival traces and workload protocol are reported separately.

Broad deployment measurements use an explicit residual-verification accounting rule,

$$C_{\mathrm{exposed}} = \max(C_{\mathrm{verify}} - \Delta_{\mathrm{slack}}, 0), \tag{11}$$

with 2,000 ms slack for Qwen/Mistral and 6,000 ms for the broad Llama protocol. All new query-qualified results use the directly measured complete service clock with zero slack. Component RPC sums provide a work breakdown alongside the directly measured wall clock.

D. Physical Accounting and Reproducibility

At initial scheduler allocation, actual prompt IDs and native hit count h_r give $n_r - h_r$ newly required prefill tokens, with separate verification counters. These measure work; latency is measured directly.

Source groups and policies are frozen before opening each independent cohort. Source-cluster statistics keep tenant/query repetitions nested and retain process variation. The artifact supplies request-level data, model/runtime manifests, source snapshots, aggregation code and archive checksums.[1]

VI. Evaluation

We test whether governing layout, evidence and publication creates useful shared service state: first complete-cost performance, then behavior, governance and operating conditions.

A. Workloads, Controls, and Units

The primary cohort contains three HotpotQA and three RepoBench sources, chosen by frozen hash order after excluding 567 previously used sources [9], [10]. Reading-comprehension data come from the LongBench collections [11]. Each source defines a ten-tenant, ten-round template with three recurring query wrappers and changing private metadata. This is an explicitly repeated authorized directory, not a measured production query distribution.

We run Qwen3-8B, Mistral-7B-Instruct-v0.3, and Llama-3.1-8B-Instruct [12]–[14]. Both policies have the same 16K window, retained task content, greedy decoding, fixed seed, request order, and two-request wave clock. HotpotQA has a 128-token generation budget and RepoBench 384. *Raw* retains the original order with native APC; *ContractKV* uses the frozen candidate, eligibility, margin/EOS checks, and publication path.

[1] https://github.com/zhouz4884-coder/contractkv. Performance evidence: `84dd0366`. Protocol-specific freezes, additional evidence and hashes are in the supplement register.

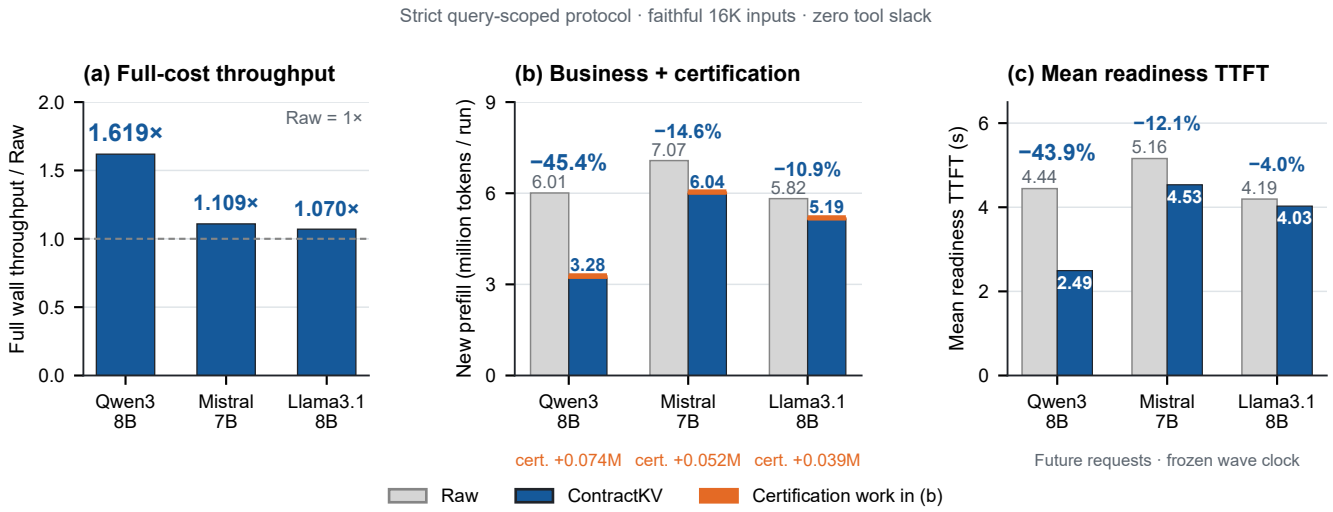


Fig. 2. Strict query-qualified performance on six source-disjoint HotpotQA/RepoBench templates, three processes/model, faithful 16K inputs, and zero slack. (a) Throughput charges all business responses, qualification, failed attempts, and synchronous waiting. (b) Actual business plus qualification new-prefill counts per run; orange caps mark certification work at its true scale, with exact added counts below. (c) Mean readiness TTFT over future requests uses the frozen two-request wave clock. Three-repeat values and tail statistics are reported separately.

Mistral and Llama inherit the development-frozen RepoBench L0 route.

Three fresh processes/model/policy yield 10,800 business generations and 4,860 future pairs: 600 business requests/run, including 540 future requests. Six sources are the independent content units; tenant, round and process observations remain nested. Ranges are process min–max; intervals resample source clusters and repetitions.

We retain prompt/output IDs, task scores, routes, qualifications, timing, and cache counters. QA uses the frozen token-F1 scorer; code uses the existing SequenceMatcher-based edit-similarity adapter. These are declared benchmark adaptations. Material score changes use $|\Delta s| > 0.03$; exact ID agreement is measured separately. The supplement gives all six sources' utility, compiled coverage and paired tails, plus process values and clock definitions.

B. Does Governed Reuse Repay Qualification?

We expect a net gain when enough admitted requests reuse a stable prefix to repay first-use qualification. Figure 2 tests this on the same source set, with no tool slack and successful and failed attempts included. Throughput improves by 1.619× on Qwen, 1.109× on Mistral, and 1.070× on Llama. The process ranges are 1.615–1.623×, 1.107–1.110×, and 1.068–1.072×. These are complete service gains rather than prefill-only estimates.

Figure 2(b) measures the work saved: including qualification, total new-prefill counts fall by 45.44%, 14.64%, and 10.89%. These scheduler observations are workload counts, not inferred GPU time. Directly measured mean readiness falls by 43.87%, 12.09%, and 3.99%.

The throughput result does not imply lower tails. All p99 point estimates increase (Table IV); Mistral's source-cluster/repetition interval for p99 reduction is wholly negative, [-60.576, -0.137]%. Mean reduction intervals are [5.640, 79.479]%, [-0.350, 34.781]%, and [1.550, 5.724]%. The cohort therefore supports a full-cost execution benefit with source-dependent latency effects.

TABLE IV
TAIL COST AND ADMITTED COVERAGE IN THE PRIMARY COHORT. READINESS STATISTICS CONCERN FUTURE REQUESTS. FULL PROCESS RANGES AND SOURCE-CLUSTER INTERVALS ARE IN THE SUPPLEMENT.

Model	Compiled / future pairs	p99 raw → CKV (ms)
Qwen	684 / 1,620	8,561.8 → 9,146.6
Mistral	342 / 1,620	9,418.2 → 9,511.3
Llama	270 / 1,620	7,851.3 → 7,905.7

A second independent six-source Qwen cohort reaches 1.803× throughput (process range 1.798–1.808×) with 774/1,620 compiled future pairs. Its results remain separate from Fig. 2.

Finding: admitted prefix reuse repays qualification in the task-preserving zero-slack cohort, with directly observed reductions in prefill work and complete service time.

a) Breadth across deployment configurations.: Table V complements the task-preserving result with four deployment configurations. The recorded 29.41–44.69% TTFT band uses 2-s/6-s residual slack. A frozen-input, three-repeat replay charges the complete controller with zero slack and measures service wall directly. Qwen3-14B TP2 retains a 20.93% full-cost TTFT reduction and 1.109× wall throughput; Mistral measures 0.994×. Qualification therefore needs sufficient authorized reuse to repay its complete cost. All attempted and aborted work is charged.

C. Does Qualification Preserve Observed Behavior?

Compilation must obtain its gain within the admitted behavioral scope. Figure 3 reports both scope coverage and free-running evidence. All 1,296 compiled future pairs have identical output IDs and unchanged scores. Including initial business

TABLE V
FOUR DEPLOYMENT CONFIGURATIONS: RECORDED RESIDUAL-SLACK ACCOUNTING AND A FROZEN-INPUT, THREE-REPEAT COMPLETE-COST REPLAY. *Recorded* MEAN-TTFT REDUCTIONS USE THE ORIGINAL RESIDUAL-VERIFIER CONVENTION (2-S SLACK; LLAMA 6 S). *Full* CHARGES MEASURED COMPLETE CONTROLLER COST WITH ZERO SLACK; *wall* IS DIRECT FULL-SERVICE WALL (RAW/CKV). EACH REPLAY USES A FRESH SERVER PROCESS; RECORDED AND REPLAYED COLUMNS RETAIN THEIR OWN CLOCKS.

Configuration	Recorded TTFT ↓	Full-cost TTFT ↓	Direct wall ×	Recorded prefill ↓
Qwen3-8B	37.81%	10.82%	1.020	59.13%
Qwen3-14B TP2	44.69%	20.93%	1.109	60.57%
Mistral-7B	31.72%	2.00%	0.994	39.46%
Llama-3.1-8B	29.41%	−0.20%	1.012	30.97%

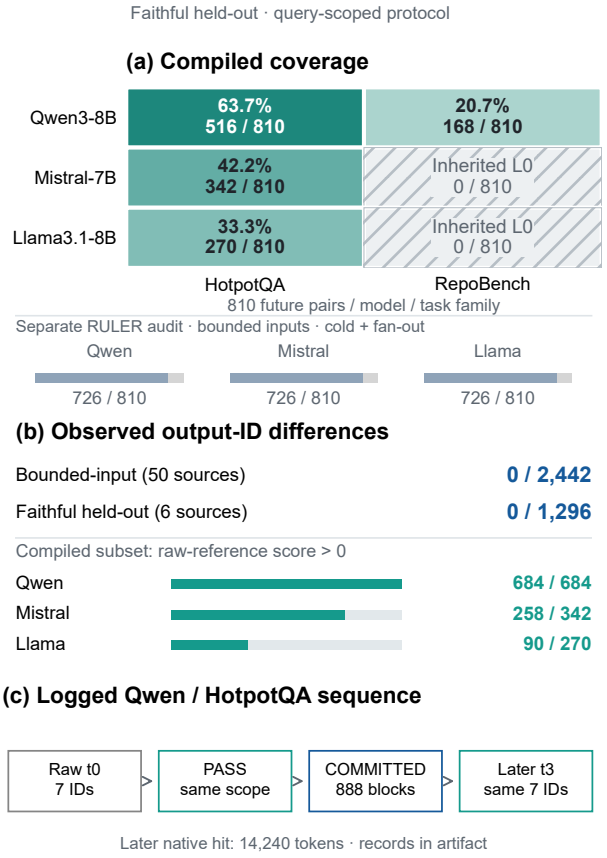


Fig. 3. Qualification coverage and free-running evidence. (a) Each model/task cell has 810 future pairs; hatched cells inherit L0. The separate RULER strip uses bounded inputs and pooled cold/fan-out traces, not the faithful matrix protocol. (b) Output-ID differences and raw-reference positive-score counts within compiled subsets. Positive credit is not complete correctness. (c) One logged Qwen/HotpotQA witness, PASS, committed prefix and later identical output; full records are archived.

rounds gives 1,440 compiled pairs with the same outcome. HotpotQA contributes compiled requests for every model; Qwen additionally admits RepoBench, whereas Mistral/Llama keep the inherited L0 route.

Behavioral preservation and absolute utility remain different outcomes. Qwen's 684 admitted raw references all receive positive score credit; the counts are 258/342 for Mistral and 90/270 for Llama. Llama preserves 180 zero-F1 future references from two HotpotQA sources, and Mistral preserves 84 from one source. Neither exact agreement nor positive partial credit establishes semantic correctness.

All-route material increases/decreases are 0/0, 2/1 and 5/3 for Qwen/Mistral/Llama, all on L0. Scheduled raw-only repeats contain corresponding output variants, showing background variation while leaving policy-associated execution effects unresolved.

The separate 50-source bounded-input audit retains 2,442 compiled future pairs: 2,178 RULER, 174 NarrativeQA and 90 RepoBench [15]–[17], with zero compiled ID changes and 33 all-route decreases on L0. Each model's RULER coverage is 726/810 (210/270 cold, 516/540 fan-out). Figure 3 separates this fixed-cap, mixed cold/fan-out protocol from the faithful non-RULER matrix.

a) Why the scope matters.: A broad-wrapper development protocol admitted later wrappers and produced 108 material decreases among 3,672 compiled pairs. A Qwen NarrativeQA development case changed its raw answer under an allowed turn-field change while the compiled answer retained the witness, despite a 0.40625 candidate margin. The evaluated eligibility class was frozen before the independent sources were opened. A margin alone cannot justify extending evidence to unseen dynamic inputs.

b) Actual output horizon.: Separate conditions produce and verify naturally terminated outputs of 642 tokens for Qwen, 769 for Mistral, and 578 for Llama, each with six correct future pairs across three processes. They use explicitly declared 128/128/192-item schemas. Llama's 224-item condition omits the final 100 items and remains a task failure. These results establish actual 512+ output conditions, not universal long-output correctness.

A common-scope HotpotQA study uses the same 30 sessions across three models and three repetitions, yielding 2,430 future pairs. Business new-prefill reductions are 48.05%, 40.69% and 38.38% for Qwen, Mistral and Llama. Its zero-slack macro mean TTFT reduction is 5.242% (source-cluster 95% interval $[-2.162, 12.564]$%); the supplement gives the timing variants and the original RULER statistical analysis.

Finding: qualified reuse preserves observed free-running behavior on admitted requests across synthetic and non-RULER tasks; coverage and reference utility identify the scope of that evidence.

D. Does Governance Preserve the Service Benefit?

Figure 4 isolates governance from layout opportunity. D1 uses a prequalified layout without online registration; D2 adds verification and transactional registration. At the same 500-ms SLO, native D0, D1 and D2 deliver 0.066, 1.472 and 1.428 requests/s. D2 retains 97.0% of D1 goodput, paying 0.044 requests/s (3.0%) for governance; TTFTs are 194.95/194.72 ms. D0 supplies context and D1 the layout-matched comparison.

The value of admission. In the independent three-model, five-challenge study, 102 of 180 dependency-valid candidates change free-running output IDs or termination; all 102 are rejected. The remaining 78 unchanged candidates are admitted, and rejected candidates publish zero new candidate blocks

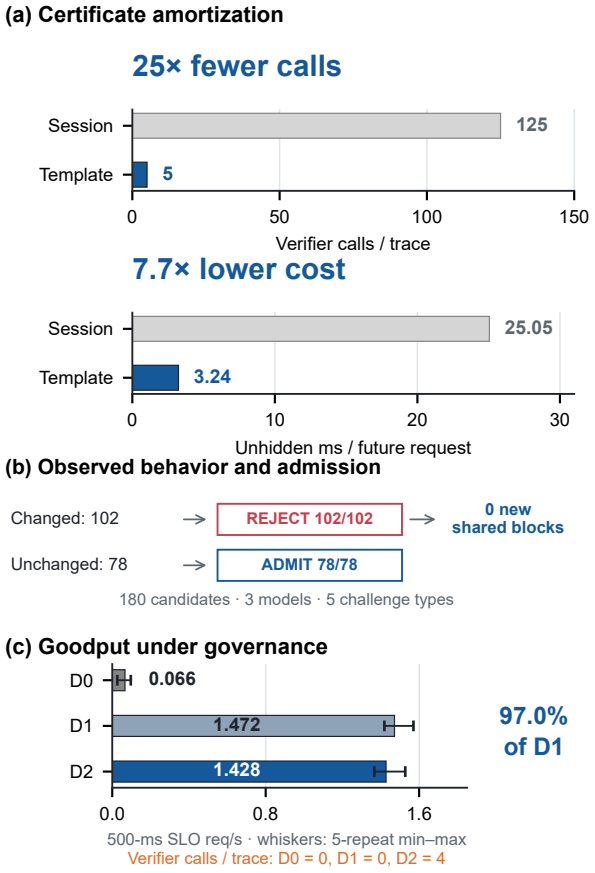


Fig. 4. Separate deployment controls. (a) Registration granularity with fixed layout/locality; three processes and 1,125 future requests/trace. (b) Three models and five challenge types: all 102 behavior-changing candidates are rejected; all 78 unchanged candidates are admitted; rejected candidates add zero shared candidate blocks. (c) Same-engine D0/D1/D2 500-ms SLO goodput; whiskers are five-process min–max, not confidence intervals.

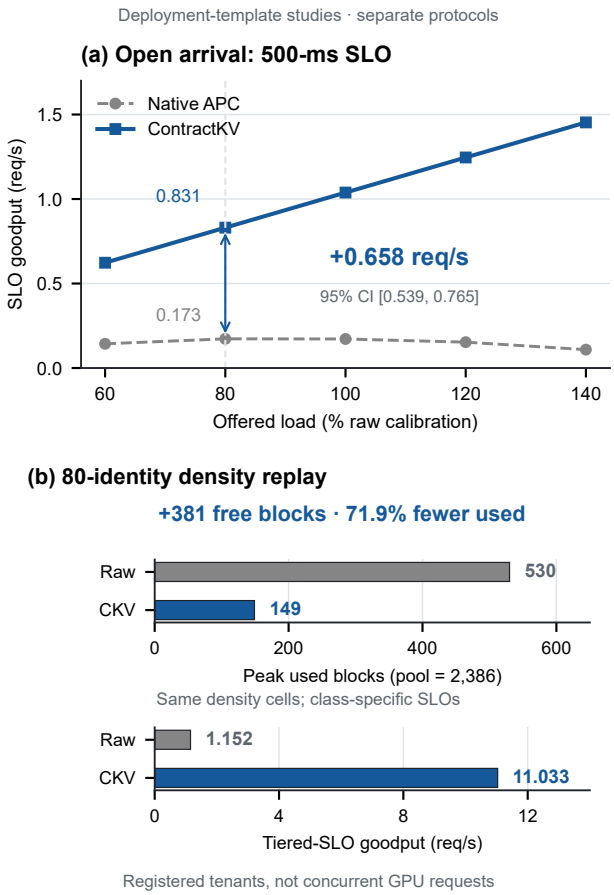


Fig. 5. Distinct deployment protocols. (a) Open-arrival 500-ms SLO goodput, with the archived hierarchical-bootstrap interval for the absolute increase at 80% load; 4.81× is that point's ratio. (b) The same 80-identity density cells supply occupancy and goodput under class-specific 750/1,500/3,000-ms SLOs, with a 2,386-block pool. Registered identities are not simultaneous GPU requests.

(Fig. 4b). The changed subset includes all 30 material score decreases. Eighteen decisions reject before a verifier RPC. Separate live cancellation, duplicate, timeout and scope controls exercise the PSTP publication boundary; the 325,000-operation stress test exercises an in-memory state machine.

1) The unit of amortization: Holding the qualified layout and locality fixed isolates the registration unit. Session-level registration invokes the verifier 125 times per trace; template-level registration invokes it five times, reducing unhidden verification exposure from 25.05 to 3.24 ms/future request (Fig. 4a). Prefix hits remain 97.82%/98.19%. The service object shares evidence cost while retaining the qualification and publication boundary.

Query-qualified execution has a different amortization unit. The second Qwen cohort obtains 774 compiled future pairs from 27 issued scope-repeat decisions and 42 total GPU calls, including failures. Its cost grows with distinct selected scopes, then amortizes across tenants using them. This preserves the template prefix as the physical object without assigning deployment's $O(1)$ behavioral cost to arbitrary dynamic queries.

At 25 tenants/template and 80% calibrated arrival load, deployment 500-ms goodput rises from 0.173 to 0.831 requests/s: +0.658, with a hierarchical-bootstrap 95% interval [0.539,0.765] (Fig. 5). Across 10–80 registered identities, bounded-concurrency density replay sustains 9.408–11.072 tiered-SLO requests/s versus native APC's 0.900–1.152 (Supplementary Table S-III). At 80 identities, the same cells use 71.9% fewer blocks, leaving 381 additional free blocks. They jointly measure consolidation and service benefit without isolating headroom as the sole cause. These marker-bearing deployment traces have a separate task-quality audit from the primary cohort.

Finding: publication governance retains the frozen layout's service benefit, and the template object supports measurable cost sharing and physical consolidation across tenants.

2) Physical authorization on every route: Authorization must survive raw fallback. Nine fresh-server cells (three models, three repetitions) each exercise six real-task contexts and one marker context without clearing the cache. The 828 business requests comprise 702 real-task and 126 marker controls.

Legacy cross-domain and old-binding/new-epoch raw fallback each hit the prior prefix in 63/63 controls. Domain/epoch-bound block hashes prevent both first-request hits while retaining authorized same-domain sharing (Table VI). The authority denies all 63 stale identities before dispatch; 93 rejected/abstained qualifications add zero candidate tokens.

TABLE VI
ALL-ROUTE AUTHORIZATION INTEGRATION, THREE MODELS AND THREE PROCESSES. EACH PHYSICAL CHECK HAS 63 CONTROLS: SEVEN CONTEXTS × NINE CELLS. REPEATS SHARE CONTEXTS. THE ENGINE IS NOT CLEARED BETWEEN CHECKS.

Sharing condition	Native hit / controls
Same domain/epoch, same principal repeat	63/63
Same domain/epoch, second authorized principal	63/63
Different domain, first request	0/63
New active epoch, first request	0/63
New domain/epoch, own subsequent repeat	63/63 each

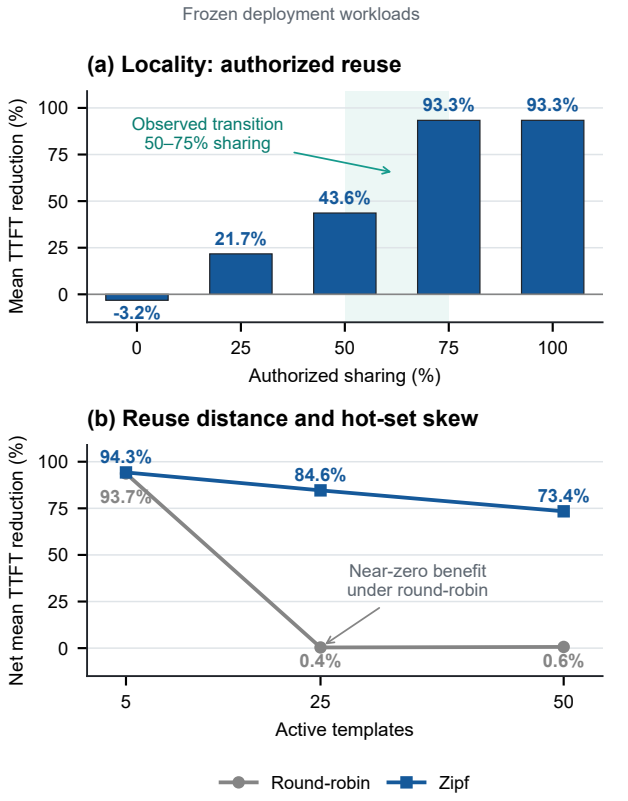


Fig. 6. Operating region in frozen deployment workloads. (a) The shaded 50–75% sharing interval is an observed transition, not a universal threshold. (b) Access-pattern sensitivity to reuse distance and hot-set skew. Both axes report TTFT reduction; neither is a direct residency fraction.

Physical sharing therefore follows authorization through every route.

E. What Determines the Operating Region?

The object pays off when authorized reuse is sufficiently dense and its physical state remains available. Figure 6 varies these conditions with a frozen deployment workload. Benefit rises sharply over an observed sharing interval, but round-robin access to a larger working set nearly eliminates it. A qualified object still needs resident physical blocks. In the primary cohort, 189 compiled business requests have incomplete hits: 135 Llama zero hits, 42 Mistral zero hits, and 12 Qwen partial hits. All 1,440 compiled business requests match the certified prefix IDs and commit boundary; each incomplete-hit request has a later full-hit peer in its two-request wave.

TABLE VII
FOUR SEALED OPERATING POINTS UNDER THE DEVELOPMENT-FROZEN BENEFIT GATE. NET BENEFIT IS PER CERTIFICATE; REJECTED POINTS SERVE RAW. CANDIDATE DIAGNOSTICS WERE ALSO RUN FOR REJECTED POINTS, SO THIS TABLE DOES NOT MEASURE AVOIDED ONLINE VERIFIER CALLS.

Prefix	Uses	Slack (s)	Route	Net benefit (s)
1500	8	3.5	Raw	0.00
3000	12	0.5	CKV	7.04
5000	10	1.5	CKV	10.84
3500	6	6	Raw	0.00

Three known-source probes replay archived IDs with read-only lookup/publication/reclamation hooks. Among 138 business requests and four qualifications, all 13 incomplete hits follow deletion of the last matching native mapping at reference count zero; salt/hash chains still match. All 12 immediate post-commit controls hit fully. This diagnoses native reclamation [18] in those sources, not every historical miss or serving performance. Figure 2 charges actual recomputation.

The frozen benefit gate selects two of four sealed points (Table VII); both selected points are net-positive at 7.04 and 10.84 s/certificate. Rejected points serve raw, while their candidate outcomes remain diagnostics. This validates the declared operating-point rule, not a learned predictor or a claim that diagnostic verifier work was avoided.

a) Sensitivity to input and cost profiles.: The full-cost benefit depends on qualified coverage and faithful inputs. Preserving HotpotQA evidence and RepoBench indentation raises raw HotpotQA F1 from 0 to 0.22222 and raw code score from 0.12067 to 0.36667 in a fixed four-source development comparison; admitted coverage increases from 30/360 to 112/360 and complete-wall throughput from 1.039× to 1.117×. Those are input-construction, not reordering-induced accuracy gains. A separate length-aware cost-profile variant lowers one Qwen p99 point estimate but sacrifices coverage and throughput; the negative tradeoff and its uncertainty remain in the Supplement.

Finding: scope determines admissible sharing, and locality, residency and admission cost determine how much of that opportunity becomes service benefit.

F. What Does the External Comparator Establish?

The external comparison measures paired task-score changes against each method's native reference. Under the same $>$ 0.03 reporting threshold, a CacheBlend adapter records 224 decreases and 331 increases in 2,250 pairs against its own separator-matched reference; ContractKV records two decreases and no increases in 2,250 pairs against its own raw APC reference. These reference-relative distributions characterize each adapter; execution stacks and inputs differ. Supplementary Table S-VIII reports native baselines and threshold sensitivity. *Finding:* good average task scores and absence of harmful per-request changes are distinct outcomes; compare them against the actual reference each method uses.

VII. RELATED WORK

Physical and cross-position reuse. PagedAttention provides block-based KV memory management and sharing [2];

SGLang's RadixAttention organizes reusable prefixes for structured language-model programs [3]. Prompt Cache exposes reusable modules through explicit schemas [4]. CacheBlend selectively recomputes information when combining cached passages [5], while EPIC uses position-independent composition with LegoLink [6]. CacheSlide and MiniPIC further explore cross-position or position-independent reuse [19], [20]. Liu et al. reorder table rows and fields to increase prefix reuse in analytics [8]. These systems expand the execution mechanisms and units available for reuse. ContractKV couples a compiled representation to a reusable service contract: behavioral evidence governs admissibility, PSTP governs visibility, and the Service-Template Certificate governs authorization and lifecycle. Supplementary Table S-VI compares these responsibilities without inferring unreported safety capabilities of other systems.

Application and service scheduling. Parrot's Semantic Variables expose application dataflow to the serving system [7]. Llumnix dynamically schedules serving requests across instances [21]; Sarathi-Serve addresses prefill/decode throughput–latency interference [22]; and Preble combines prompt reuse with distributed load balancing [23]. DistServe and Mooncake separate serving stages or distribute KV storage and computation [24], [25]. These mechanisms govern resource placement and execution. ContractKV instead determines when a compiled representation is eligible for publication and authorized reuse within one cache domain. Its waiting and residency measurements identify where model-backed qualification and native scheduling interact.

Cost models, contracts, and cache privacy. Vidur combines experimental profiling with predictive models to estimate serving behavior [26]; our frozen operating-point and cost-profile studies expose the effect of profiling support on useful admission. Classical service contracts such as WSLA associate measurable service obligations with monitoring and enforcement [27]. ContractKV binds a reusable model-execution artifact to evidence, identity, authorization, and a transactional boundary. Work on prompt leakage and audits of prompt-cache sharing demonstrates that cache timing is itself an observable privacy channel [28], [29]. Those results motivate separating authorization-controlled cache state from protection against timing inference.

VIII. Discussion and Limitations

A. Behavioral Evidence and Reuse Density

Token-path certification supplies execution-scoped behavioral evidence. Figure 3 reports output agreement and raw-reference task utility separately. Query-qualified sharing assumes a repeated authorized directory and the declared metadata-only observation class; evolving histories and new tool evidence require additional qualification. With many unique queries, qualification has little opportunity to amortize; production query-repeat frequencies remain unmeasured.

B. Trust Model and Manifest Registration

The operator defines template ownership, segment types, dependencies, sharing domains and versions. The trusted controller validates authority and lifecycle, and the engine enforces the associated cache identity and PSTP publication boundary. The all-route integration carries this identity through raw fallback (Table VI); engine access must be mediated by the trusted application.

The registration helper checks issuer/role, segment identity, authentication tags, content hashes, version/domain fields and consistency between declared stability, visibility and publication. Its nine-case CPU audit rejects seven malformed or unauthorized declarations, accepts a valid case, and accepts a trusted semantic mislabel. A separate 19-case client-binding audit rejects 17 malformed, expired or mismatched bindings. These are declaration and client checks; the helper itself is not integrated into the live endpoint. The tested internal authority does not constitute a public tenant-IAM gateway.

The remaining semantic trust is concrete. If an operator labels tenant A's private `TOOL_OBS` as stable and domain-shared, a valid hash and HMAC authenticate the erroneous declaration. A live marker control publishes that declared prefix in 9/9 cases. Hashes establish byte identity, and HMACs establish binding authority; confidentiality classification remains the operator's responsibility. Epoch separation denies stale identities without erasing all old blocks or retracting in-flight requests.

Authorized sharing can also reveal cache-hit timing within a domain [28], [29]. ContractKV does not implement a timing-channel defense. P4 governs cache-state authorization; comprehensive application confidentiality requires additional controls.

C. Service and Platform Conditions

Residency and queueing determine whether qualification becomes service benefit. The independent fixed-arrival task cohort in the supplement has 24 completed main cells and low compiled coverage; its four Goodput-change intervals cross zero. Its neutral results define a measured operating limit alongside the positive deployment operating points in Fig. 5. The earlier 5/54-cell arrival pilot remains a separate partial record.

The inherited Python guard can exclude legal next-line fragments; RepoBench scores use the declared adapter. Native probes diagnose specific reclamation events, and the vLLM 0.25.1 interface audit is not a port. Distributed services need coordinated lifecycle/publication. Qualification is economical where authorized reuse and resident state repay its service-clock cost.

IX. Conclusion

ContractKV turns a structurally reusable prompt prefix into a governed multi-tenant KV service object. CALC exposes a legal physical prefix, scoped behavioral evidence determines its admissible use, PSTP controls first shared visibility, and a versioned Service-Template Certificate lets authorized consumers share qualification. The contribution lies in making these decisions one executable service lifecycle.

The measurements connect that lifecycle to useful service outcomes. Under task-preserving zero-slack query qualification, three models achieve 1.070–1.619× complete-service

throughput, with identical raw output IDs on all 1,296 admitted future pairs. Fixed-layout template sharing reduces verifier calls from 125 to five and unhidden exposure from 25.05 to 3.24 ms/request; an evaluated open-arrival deployment point gains 0.658 SLO-compliant requests/s. The four-configuration complete-cost results in Table V and the all-route physical authorization controls identify the costs and enforcement boundary behind those gains.

Three lessons follow: compilation creates reuse opportunity; evidence and transactional publication make it an admissible shared resource; and template-level qualification sharing, together with locality and residency, determines its service economics. Treating reusable KV as an evidence-bound service object provides a concrete basis for deploying transformed prompts as shared infrastructure.

REFERENCES

[1] Azure Samples, "Azure AI Search + Azure OpenAI demo: Conversation builder and answer template," source revision `558ffd397edd`, *azure-search-openai-demo*. [Online]. Available: https://github.com/Azure-Samples/azure-search-openai-demo. Accessed: Oct. 17, 2026.

[2] W. Kwon *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. ACM SIGOPS Symp. Oper. Syst. Principles (SOSP)*, 2023, pp. 611–626, doi: 10.1145/3600006.3613165.

[3] L. Zheng *et al.*, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[4] I. Gim, G. Chen, S.-s. Lee, N. Sarda, A. Khandelwal, and L. Zhong, "Prompt Cache: Modular attention reuse for low-latency inference," in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 6, 2024.

[5] J. Yao *et al.*, "CacheBlend: Fast large language model serving for RAG with cached knowledge fusion," in *Proc. 20th Eur. Conf. Comput. Syst. (EuroSys)*, 2025, doi: 10.1145/3689031.3696098.

[6] J. Hu *et al.*, "EPIC: Efficient position-independent caching for serving large language models," in *Proc. 42nd Int. Conf. Mach. Learn. (ICML)*, PMLR, vol. 267, 2025, pp. 24391–24402.

[7] C. Lin, Z. Han, C. Zhang, Y. Yang, F. Yang, C. Chen, and L. Qiu, "Parrot: Efficient serving of LLM-based applications with semantic variable," in *Proc. 18th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, 2024, pp. 929–945.

[8] S. Liu *et al.*, "Optimizing LLM queries in relational data analytics workloads," in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 7, 2025. Author version: arXiv:2403.05821.

[9] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning, "HotpotQA: A dataset for diverse, explainable multi-hop question answering," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2018, pp. 2369–2380, doi: 10.18653/v1/D18-1259.

[10] T. Liu, C. Xu, and J. McAuley, "RepoBench: Benchmarking repository-level code auto-completion systems," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[11] Y. Bai *et al.*, "LongBench: A bilingual, multitask benchmark for long context understanding," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2024, pp. 3119–3137, doi: 10.18653/v1/2024.acl-long.172.

[12] A. Yang *et al.*, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

[13] A. Q. Jiang *et al.*, "Mistral 7B," *arXiv preprint arXiv:2310.06825*, 2023.

[14] A. Dubey *et al.*, "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[15] C.-P. Hsieh *et al.*, "RULER: What's the real context size of your long-context language models?" in *Proc. Conf. Language Modeling (COLM)*, 2024.

[16] T. Kočiský *et al.*, "The NarrativeQA reading comprehension challenge," *Trans. Assoc. Comput. Linguistics*, vol. 6, pp. 317–328, 2018, doi: 10.1162/tacl_a_00023.

[17] P. Dasigi, K. Lo, I. Beltagy, A. Cohan, N. A. Smith, and M. Gardner, "A dataset of information-seeking questions and answers anchored in research papers," in *Proc. Conf. North American Chapter Assoc. Comput. Linguistics: Human Language Technologies (NAACL-HLT)*, 2021, pp. 4599–4610, doi: 10.18653/v1/2021.naacl-main.365.

[18] vLLM Project, "Automatic prefix caching," *vLLM Documentation*, ver. 0.10.2. [Online]. Available: https://docs.vllm.ai/en/v0.10.2/design/prefix_caching.html. Accessed: Oct. 17, 2026.

[19] Y. Liu *et al.*, "CacheSlide: Unlocking cross position-aware KV cache reuse for accelerating LLM serving," in *Proc. 24th USENIX Conf. File Storage Technol. (FAST)*, 2026, pp. 83–99.

[20] N. Ordonez and T. Parnell, "MiniPIC: Flexible position-independent caching in $<$100 LOC," *arXiv preprint arXiv:2606.13126*, 2026.

[21] B. Sun *et al.*, "Llumnix: Dynamic scheduling for large language model serving," in *Proc. 18th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, 2024, pp. 173–191.

[22] A. Agrawal *et al.*, "Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve," in *Proc. 18th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, 2024, pp. 117–134.

[23] V. Srivatsa, Z. He, R. Abhyankar, D. Li, and Y. Zhang, "Preble: Efficient distributed prompt scheduling for LLM serving," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.

[24] Y. Zhong *et al.*, "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *Proc. 18th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, 2024, pp. 193–210.

[25] R. Qin *et al.*, "Mooncake: Trading more storage for less computation—A KVCache-centric architecture for serving LLM chatbot," in *Proc. 23rd USENIX Conf. File Storage Technol. (FAST)*, 2025, pp. 155–170.

[26] A. Agrawal *et al.*, "Vidur: A large-scale simulation framework for LLM inference," in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 6, 2024.

[27] A. Keller and H. Ludwig, "The WSLA framework: Specifying and monitoring service level agreements for Web services," *J. Network Syst. Manage.*, vol. 11, no. 1, pp. 57–81, 2003, doi: 10.1023/A:1022445108617.

[28] G. Wu, Z. Zhang, Y. Zhang, W. Wang, J. Niu, Y. Wu, and Y. Zhang, "I know what you asked: Prompt leakage via KV-cache sharing in multi-tenant LLM serving," in *Proc. Network Distrib. Syst. Security Symp. (NDSS)*, 2025, doi: 10.14722/ndss.2025.241772.

[29] C. Gu, X. L. Li, R. Kuditipudi, P. Liang, and T. Hashimoto, "Auditing prompt caching in language model APIs," in *Proc. 42nd Int. Conf. Mach. Learn. (ICML)*, 2025. Author version: arXiv:2502.07776.